

GUÍA 03: GUÍA 03: ANÁLISIS Y SELECCIÓN DE ESTILOS Y ENFOQUES ARQUITECTÓNICOS

Aplicación de Clean Architecture en un sistema Marketplace

ASIGNATURA	IS-488 ARQUITECTURA DE SOFTWARE
DOCENTE	ING. LIZBETH JAICO QUISPE
SEMESTRE	2026-II
LABORATORIO	02, 04, 05
SEMANA	3
CASO DE ESTUDIO	Marketplace de productos para mascotas
REFERENCIA	GoPet: https://www.gopet.pe/
DURACIÓN	2 horas

I. PROPÓSITO DEL LABORATORIO

En la Guía 02 analizamos el marketplace y propusimos una primera arquitectura en capas. En esta sesión damos el paso siguiente del proceso de arquitectura: decidir cómo se organizan internamente las responsabilidades y las dependencias del sistema, aplicando el enfoque de arquitectura limpia.

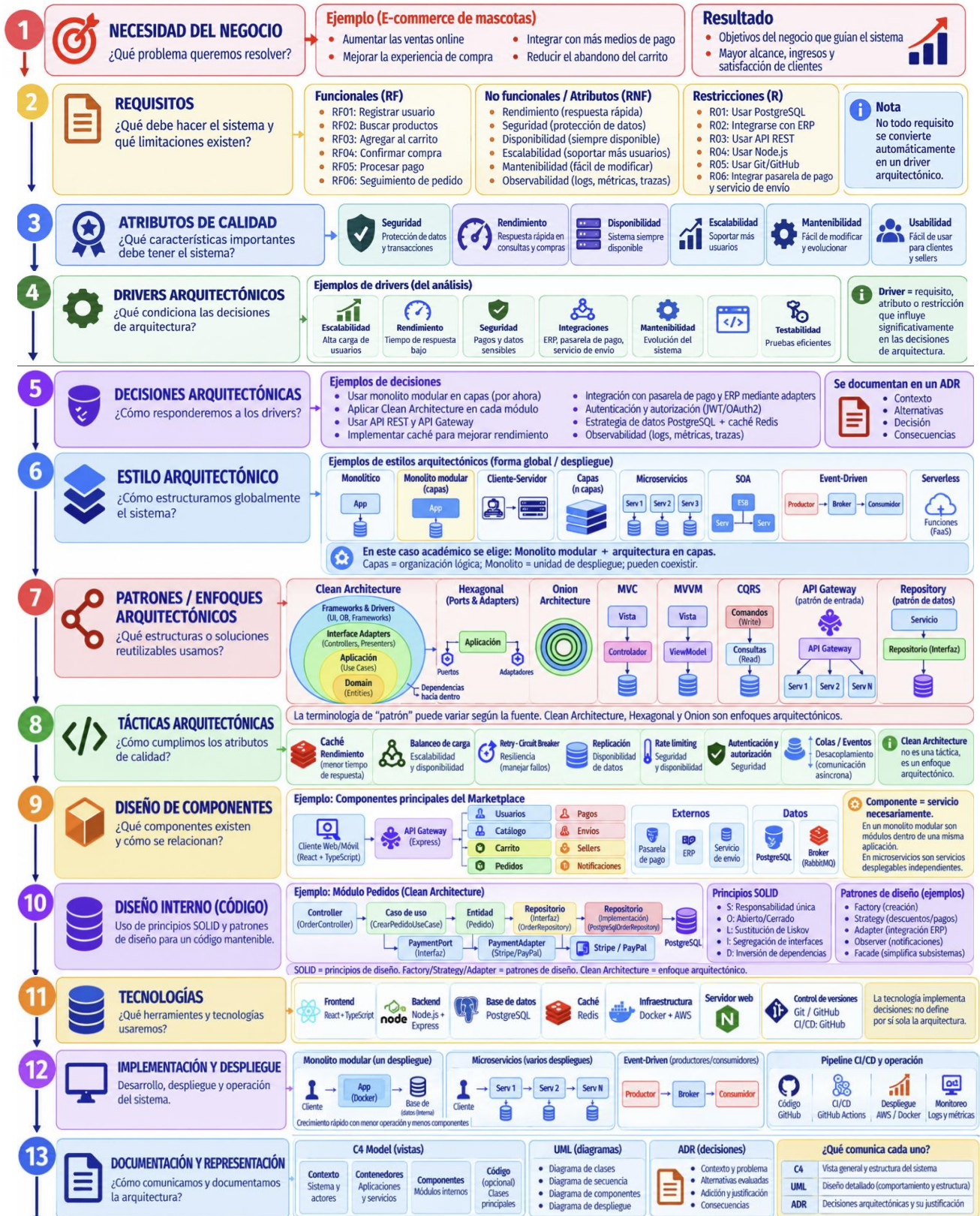
Advertencia importante: este laboratorio no es de programación. El proyecto ejemplo se abre, se lee y se ejecuta, pero no se desarrolla. Lo que se aprende es a reconocer y a decidir dónde vive cada responsabilidad, que es trabajo de arquitecto.

II. HERRAMIENTAS

Herramienta	Para qué se usa	Enlace
Node.js 18 o superior	Ejecutar el proyecto Angular y las pruebas del dominio.	https://nodejs.org/es/download
Visual Studio Code	Abrir el proyecto, navegar su estructura y leer las líneas de importación.	https://code.visualstudio.com/
Angular 18	Framework del proyecto ejemplo. Se instala solo con npm install; no se requiere saber Angular para auditarlo.	https://angular.dev
Git y GitHub	Versionar la propuesta de estructura y la decisión documentada.	https://github.com
Mermaid	Dibujar el diagrama de capas dentro del repositorio.	Extensión de VS Code: Markdown Preview Mermaid Support

III. MARCO CONCEPTUAL

N.º	ETAPA	¿QUÉ SE DEFINE?	PREGUNTA GUÍA	EJEMPLO EN MARKETPLACE	RESULTADO
1	Necesidad del negocio	Problema y objetivos del negocio.	¿Qué problema queremos resolver?	Aumentar ventas, mejorar compra e integrar pagos/envíos.	Objetivos del negocio
2	Requisitos	Requisitos funcionales, no funcionales y restricciones.	¿Qué debe hacer el sistema y qué limitaciones existen?	RF: buscar, carrito, compra. RNF: rendimiento, seguridad. R: PostgreSQL, REST, ERP.	Base de necesidades del sistema
3	Atributos de calidad	Características de calidad prioritarias.	¿Qué características importantes debe tener?	Rendimiento, seguridad, disponibilidad, escalabilidad, mantenibilidad y observabilidad.	Prioridades de calidad
4	Drivers arquitectónicos	Requisitos, atributos o restricciones que influyen significativamente en la arquitectura.	¿Qué condiciona las decisiones de arquitectura?	Alta carga, pagos seguros, integración ERP/pago/envío y evolución del sistema.	Drivers priorizados
5	Decisiones arquitectónicas	Decisiones que responden a los drivers y sus alternativas.	¿Cómo responderemos a los drivers?	Monolito modular, capas, Clean Architecture, API REST, caché y autenticación.	Decisiones + ADR (Architecture Decision Record)
6	Estilo arquitectónico	Forma global de organización y/o despliegue.	¿Cómo estructuramos globalmente el sistema?	Monolito, capas, cliente-servidor, microservicios, SOA, event-driven o serverless.	Estilo seleccionado
7	Patrones / enfoques arquitectónicos	Forma de organizar responsabilidades y dependencias.	¿Qué estructura arquitectónica aplicaremos?	Clean Architecture, Hexagonal, Onion, MVC, MVVM, CQRS; también API Gateway/Repository según el uso.	Estructura interna arquitectónica
8	Tácticas arquitectónicas	Mecanismos para responder a atributos de calidad.	¿Cómo cumplimos los atributos de calidad?	Caché, balanceo, retry/circuit breaker, replicación, rate limiting, autenticación/autorización.	Mecanismos de calidad
9	Diseño de componentes	Componentes/módulos y sus relaciones.	¿Qué componentes existen y cómo se relacionan?	Usuarios, Catálogo, Carrito, Pedidos, Pagos, Envíos, API Gateway, BD y servicios externos.	Mapa de componentes
10	Diseño interno	Estructura detallada dentro de los componentes.	¿Cómo funciona internamente cada componente?	Controller → Caso de uso → Entidad → Repositorio → Infraestructura; SOLID y patrones de diseño.	Diseño detallado
11	Tecnologías	Herramientas concretas para implementar las decisiones.	¿Con qué lo construiremos?	React + TypeScript, Node.js + Express, PostgreSQL, Redis, Docker/AWS, Git/GitHub.	Stack tecnológico
12	Implementación y despliegue	Construcción, integración, despliegue y operación.	¿Cómo lo construiremos y operaremos?	Git, CI/CD, Docker, AWS, logs y métricas; monolito vs. varios despliegues.	Sistema ejecutable y operable
13	Documentación y representación	Vistas, diagramas y decisiones que comunican la arquitectura.	¿Cómo comunicamos y mantenemos la arquitectura?	C4, UML y ADR.	Arquitectura documentada

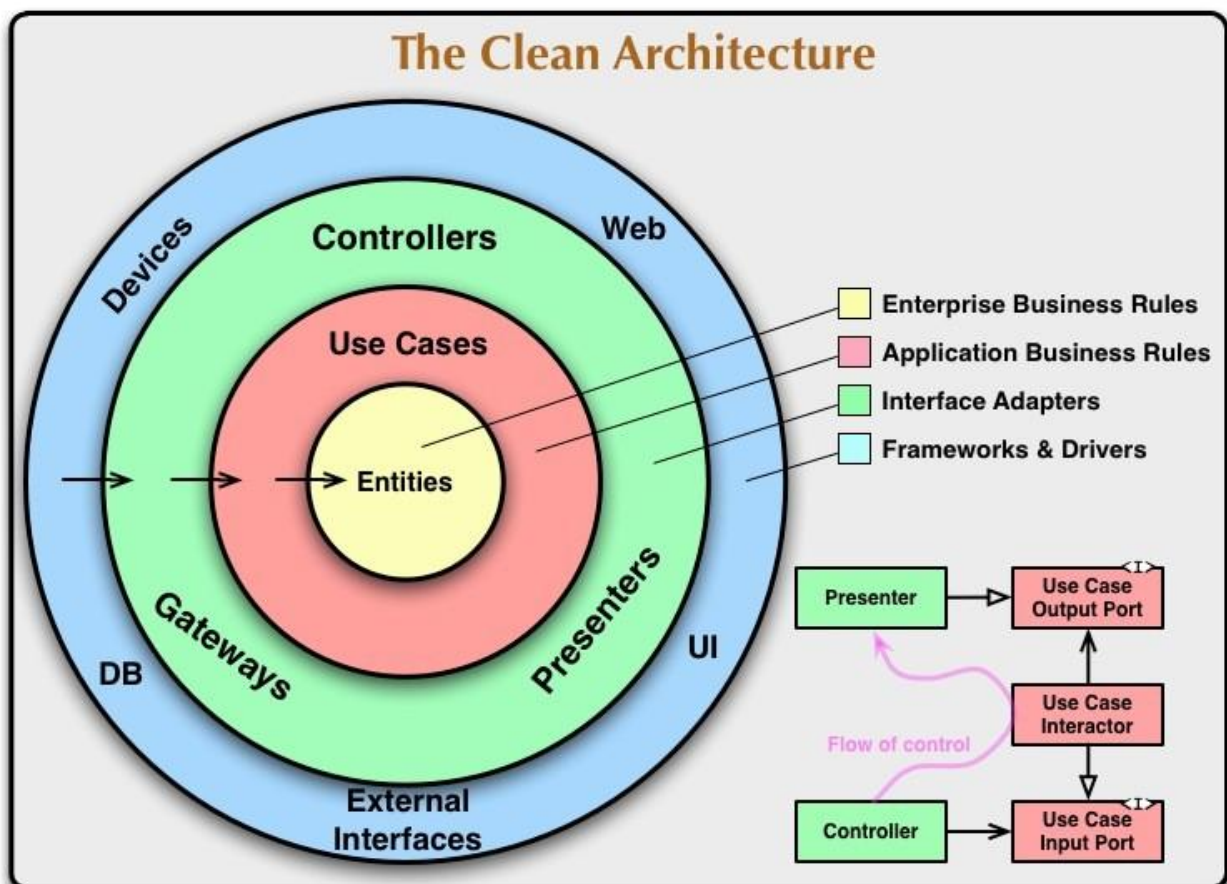


Resumen:



1. ¿Qué es Clean Architecture?

Enfoque arquitectónico que organiza el sistema alrededor de las reglas de negocio y establece que las dependencias del código deben apuntar hacia el interior, evitando que el núcleo del negocio dependa directamente de la interfaz de usuario, la base de datos, los frameworks o servicios externos.



2. ¿Cuáles son sus partes?

Tu carpeta	Capa de Clean Architecture	¿Qué contiene?	Ejemplo de GoPet
dominio	1. Domain	Entidades, objetos de valor y reglas de negocio.	Producto, Carrito, Pedido.
aplicación	2. Application	Casos de uso, puertos o interfaces de repositorio y Objeto de Transferencia de Datos cuando sean necesarios.	AgregarAlCarrito, ListarProductos, RegistrarPedido.
presentación	Presentación / adaptadores de interfaz	Pantallas, componentes visuales y, según el diseño, controladores o adaptadores de presentación.	Catálogo de productos, carrito de compras y pantalla de pago.

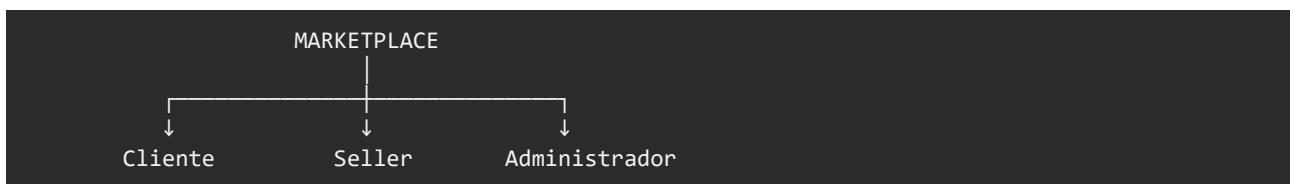
infraestructura	Infraestructura / frameworks y drivers	Implementaciones concretas para comunicarse con APIs, bases de datos y servicios externos.	Cliente HTTP, adaptador de API de productos y conexión con servicios de pago.
-----------------	--	--	---

IV. CONTEXTO DEL CASO DE ESTUDIO

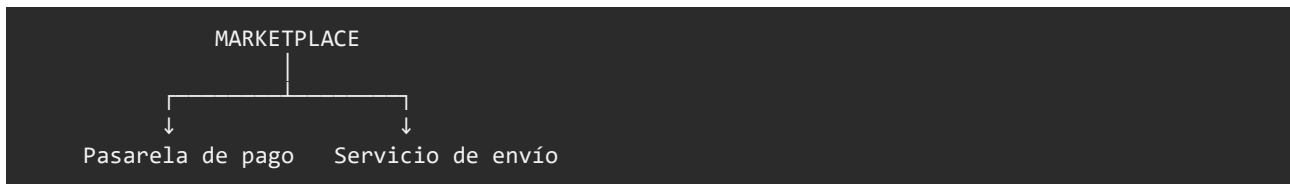
Se desea desarrollar un marketplace de productos para mascotas, en el cual diferentes sellers puedan ofrecer sus productos y los clientes puedan realizar compras.

Para comprender el contexto del negocio se tomará como referencia funcional un marketplace como GoPet.

Los principales participantes del sistema son:



También pueden existir sistemas externos:



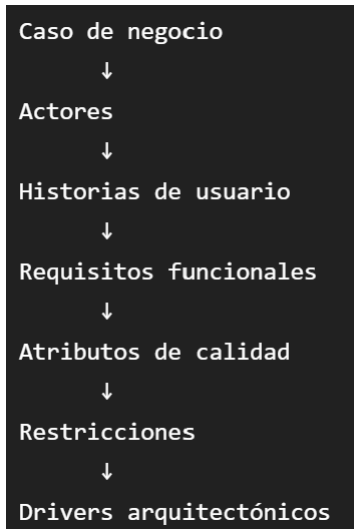
V. REPASO Y MODIFICACIÓN DE LA GUIA 2

En la Guía 02 propusimos una arquitectura en capas, como una primera propuesta. Trabajando el análisis inicial del caso de estudio de un Marketplace, trabajando en 2 etapas:

El trabajo comprendió dos etapas.

Etapas	En qué consiste	Qué se identifica
Etapas 1 Análisis del sistema	Comprender qué debe resolver el sistema antes de diseñar su estructura.	Actores, funcionalidades, historias de usuario, requisitos funcionales, atributos de calidad, restricciones y drivers arquitectónicos.
Etapas 2 Diseño arquitectónico inicial	Traducir ese análisis en una primera organización de la solución.	Responsabilidades, módulos, capas, dependencias, sistemas externos y el primer diagrama arquitectónico.

Quedando nuestro análisis de estudio:



En driver arquitectónico agregaremos DA06 : Mantenibilidad / evolución modular

ID	Driver arquitectónico	Origen	¿Por qué influye?
DA06	El sistema debe permitir modificar funcionalidades sin afectar innecesariamente otros módulos.	AC05- Mantenibilidad	Influye en la separación de responsabilidades, modularidad y dependencias internas.

Quedando también definidos nuestros drivers arquitectónicos:

Driver	Problema que plantea	Decisión que responde
DA01- Escalabilidad	Aumentarán usuarios en campañas	Monolito modular con posibilidad de escalamiento horizontal
DA02- Rendimiento	Habrà alta concurrencia	Incorporar caché y optimizar comunicación/procesamiento
DA03 -Seguridad	Hay datos sensibles	Autenticación y autorización
DA04 - Pago externo	Hay que comunicarse con una pasarela	Integración mediante API y adaptadores
DA05 -API REST	Frontend/backend deben comunicarse mediante REST	Separar interfaz y backend mediante API REST
DA06 - Mantenibilidad	Cambios no deben afectar otros módulos	Modularidad + Clean Architecture

VI. CASO PRÁCTICO

PASO 01: Abrir el proyecto

<https://github.com/devlizbethjaico/boilerplate.git>

```

# Descomprimir el proyecto y abrirlo en Visual Studio Code
cd boilerplate
code .

# Instalar dependencias (una sola vez)
npm install
  
```

PASO 3: Decisiones Arquitectónicas: Ahora que ya tenemos identificados los drivers arquitectónicos, podemos definir las decisiones arquitectónicas (ADR) que permitirán responder a las necesidades y requisitos del sistema.

ADR (Architecture Decision Record) significa Registro de Decisión Arquitectónica y documenta las decisiones importantes que tomamos durante el diseño de la arquitectura del software, junto con su justificación.

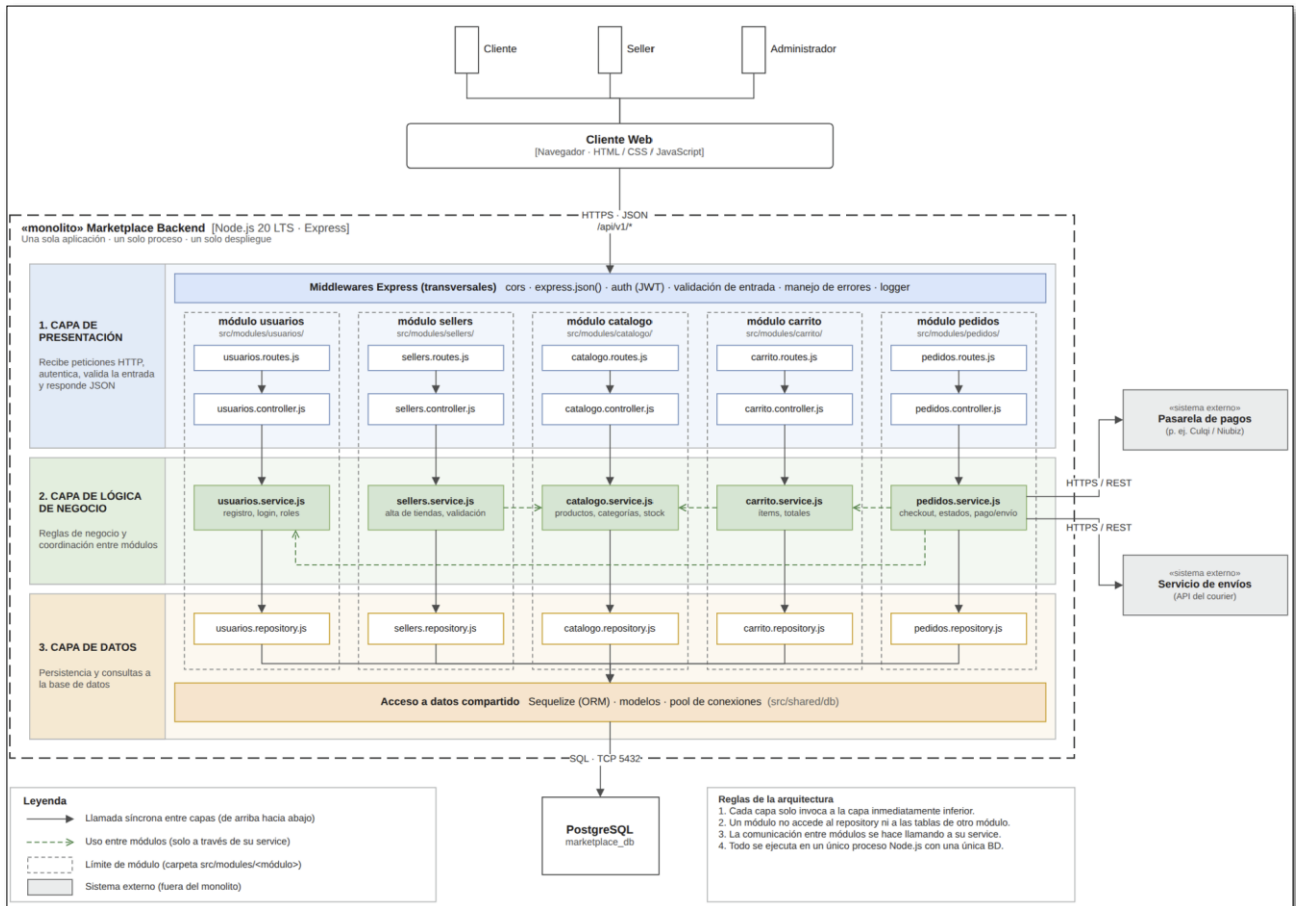
ID	Decisión arquitectónica	Driver relacionado	Justificación	Resultado
ADR-001	Monolito modular	DA01-Escalabilidad; DA06 - Mantenibilidad	Organizar las funcionalidades en módulos independientes dentro de una misma aplicación desplegable.	Módulos de Catálogo, Carrito, Pedidos, Pagos y Usuarios.
ADR-002	Clean Architecture	DA06-Mantenibilidad	Separar las reglas del negocio de los detalles tecnológicos.	Dominio, Aplicación, Infraestructura y Presentación.
ADR-003	Estrategia de caché	DA02-Rendimiento	Reducir consultas repetitivas a la fuente de datos.	Caché para información de consulta frecuente.
ADR-004	Integración de pagos mediante interfaces y adaptadores	DA04-Integración con pagos	Desacoplar los casos de uso del proveedor de pagos.	Contrato de pagos y adaptador para la pasarela externa.

PASO 4: DEFINIR EL ESTILO ARQUITECTÓNICO (ESTRUCTURA GLOBAL DEL SISTEMA)

Dentro de la carpeta arquitectura/, incorporar el documento: estilo-arquitectonico.md que describa el estilo arquitectónico seleccionado para el proyecto.

(Elaborar el diagrama de arquitectura utilizando cualquiera de las herramientas vistas en clase)

Asegurarse de que el diagrama represente los principales componentes del sistema y cómo se relacionan entre sí.



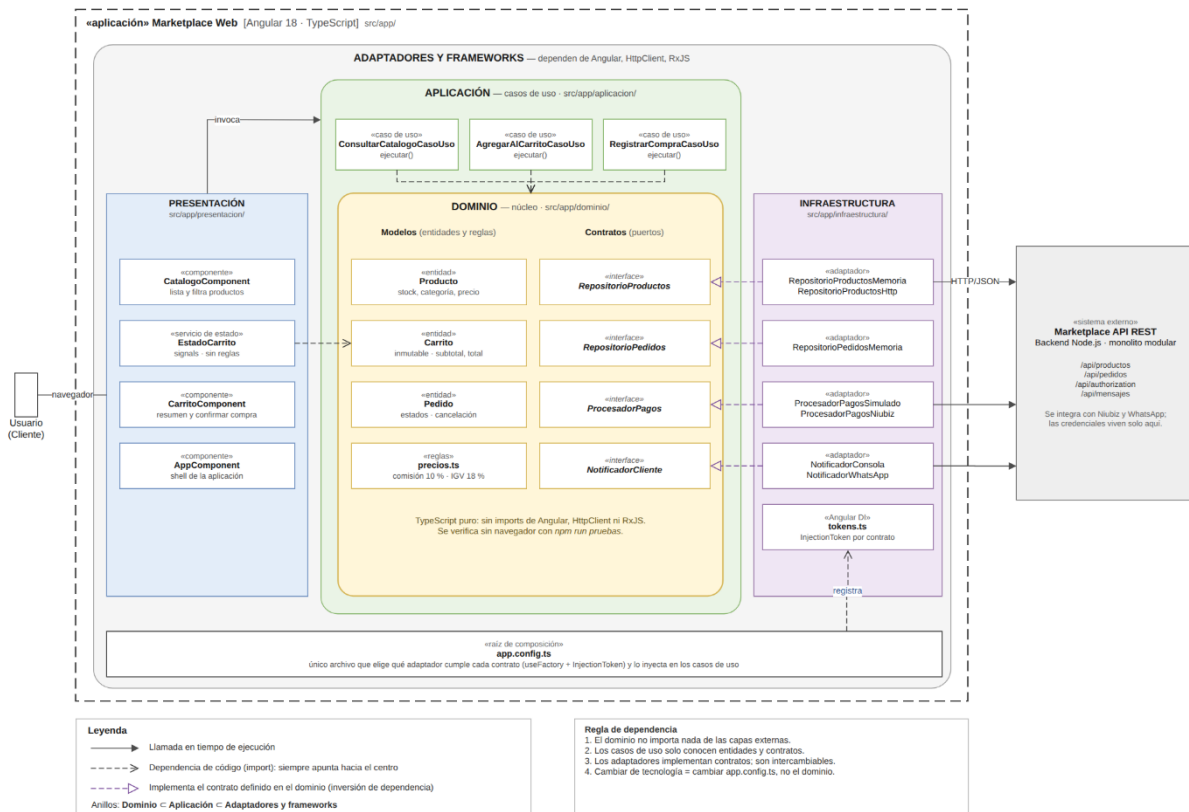
PASO 5: DEFINIR PATRONES O ENFOQUE ARQUITECTÓNICO

Dentro de la carpeta arquitectura, crear una sub carpeta enfoque y dentro crear el archivo enfoque-arquitectonico.md

Las dependencias internas mediante Clean Architecture.

Elemento	Descripción aplicada al Marketplace
Patrón / enfoque arquitectónico	Clean Architecture (Arquitectura Limpia).
Objetivo	Separar responsabilidades y controlar las dependencias hacia el dominio.
¿Qué problema resuelve?	Evita el acoplamiento entre la interfaz Angular, las reglas del negocio y las tecnologías externas, como bases de datos, API y servicios de pago.
Capas definidas	Presentación, Aplicación, Dominio e Infraestructura.
Beneficios	• Facilita el mantenimiento y las pruebas unitarias. • Permite cambiar implementaciones técnicas sin modificar innecesariamente las reglas del negocio. • Mejora la organización y separación de responsabilidades del código.

(elabora el diagrama con cualquier herramienta vista en clases)



VII. TAREA

Para la siguiente sesión, deberán tener **completados y definidos los seis primeros entregables** del proceso de arquitectura:

1. **Necesidad del negocio**
2. **Requisitos**
3. **Atributos de calidad**
4. **Drivers arquitectónicos**
5. **Decisiones arquitectónicas**
6. **Estilo arquitectónico**

Además, deberán **trabajar el enfoque arquitectónico: Clean Architecture**, identificando cómo se organizarían las responsabilidades y dependencias internas del sistema.

Importante: No se requiere desarrollar nuevas funcionalidades. El trabajo se centra en analizar, tomar decisiones arquitectónicas y justificar la propuesta para el proyecto.